

71 – Authorization

След идентификация, имаме достъп до информация и възможности какво можем и не можем да правим. Това е авторизация.

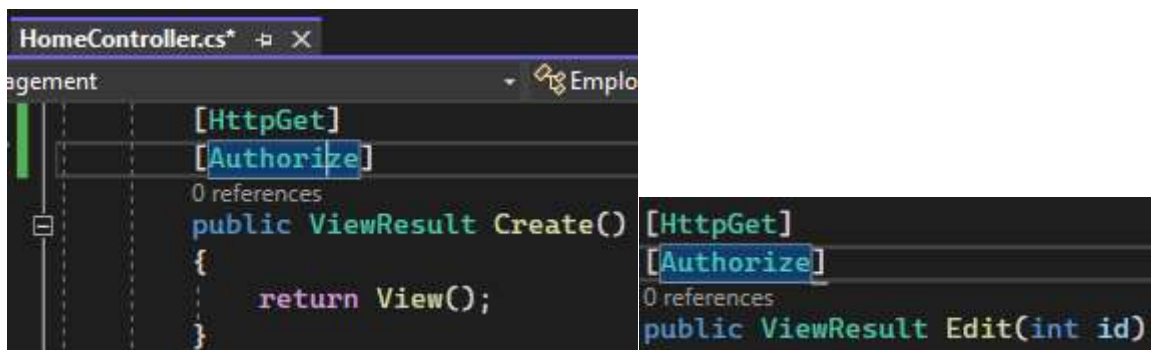
Authorization in ASP.NET Core

Authentication is identifying who the user is

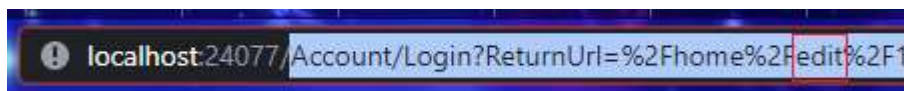
Authorization is identifying what the user can and cannot do

[Authorize] attribute controls Authorization

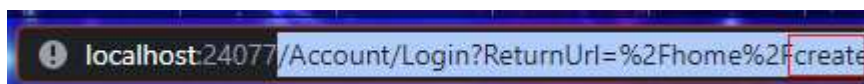
HomeController -> Index + Details actions нямат проблеми за достъп от анонимен потребител, но Create + Edit имат нужда от защита, затова ги декорираме с [Authorize] атрибут. Това е необходимо за Get + Post.



При Logout, вече имаме възможност да виждаме списъка и детайлите на служителите, но в момента, в който поискаме с бутоните да създадем или редактираме служител, ни пренасочва към линк за Login.



Account/Login?ReturnUrl=%2Fhome%2Fcreate



<https://meyerweb.com/eric/tools/dencoder/>



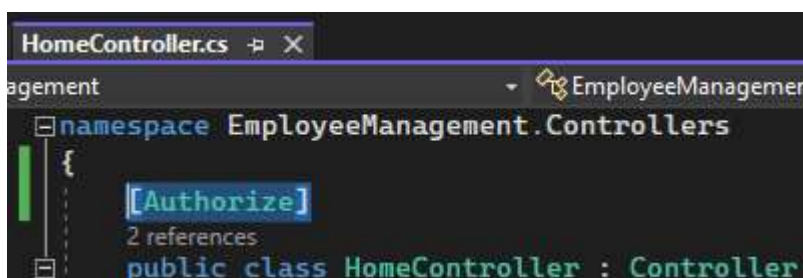
URL Decoder/Encoder

`http://localhost:24077/Account/Login?ReturnUrl=/home/create`

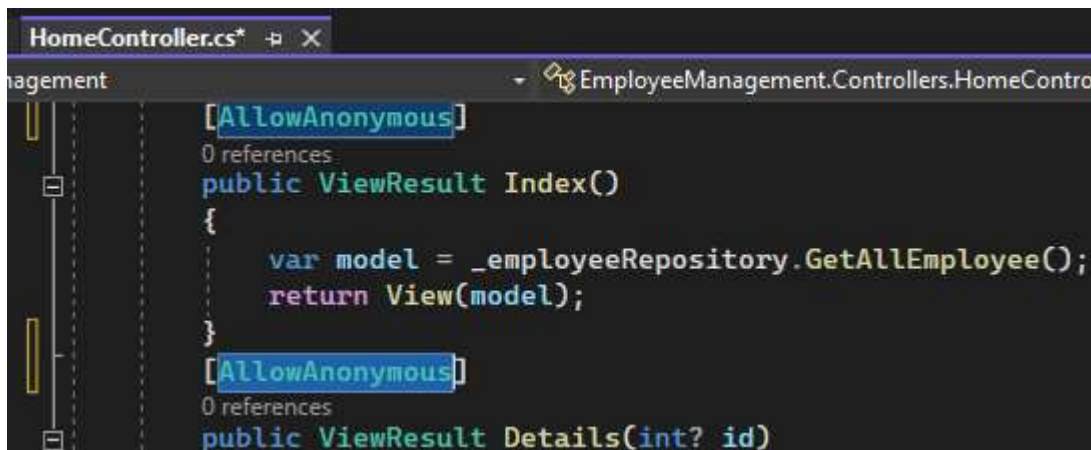
Откъдето забелязваме, че **\$2F** съответства на знака **/**

Зададен в този вид адреса с ReturnUrl= ще препрати потребителя към /home/create веднага, когато се впише.

Вместо индивидуално навсякъде да поставяме [Authorize], може да го използваме глобално в контролера, като по този начин без вписване, няма да имаме достъп до нищо от него, включително Index + Details.



Ако искаме специално Index + Details да бъдат освободени за неоторизиран достъп, ги украсяваме с [AllowAnonymous]:



На този етап сме приложили

[Authorize] – глобално в контролера

[AllowAnonymous] – индивидуално в actions.

Това няма да има същият ефект, ако ги разменим според необходимостта и оставим [AllowAnonymous] в контролера и [Authorize] в индивидуалните actions, защото тогава главния [AllowAnonymous] в контролера ще засенчи и пренебрегне вградените [Authorize].

В Startup.cs може да се използва [Authorize] глобално, вместо във всеки контролер отделно чрез вградените възможности на MVC.Authorize. Трябва да създадем нов филтър:

```
services.AddMvc(option => option.EnableEndpointRouting = false)
    .AddXmlSerializerFormatters();
```

Добавяме необходимия код с двата необходими namespace:

```
using Microsoft.AspNetCore.Mvc.Authorization;
```

```
using Microsoft.AspNetCore.Authorization;
```

```
Startup.cs  + X
- EmployeeManagement.Startup

services.AddMvc(options =>
{
    options.EnableEndpointRouting = false;
    var policy = new AuthorizationPolicyBuilder()
        .RequireAuthenticatedUser()
        .Build();
    options.Filters.Add(new AuthorizeFilter(policy));
}).AddXmlSerializerFormatters();
```

```
services.AddMvc(options =>
{
    options.EnableEndpointRouting = false;
    var policy = new AuthorizationPolicyBuilder()
        .RequireAuthenticatedUser()
        .Build();
    options.Filters.Add(new AuthorizeFilter(policy));
}).AddXmlSerializerFormatters();
```

Сайтът работи, с изключение на вписването, където виждаме грешка и повторения на Account, Login, returnUrl многократно:

localhost:24077/Account/Login?ReturnUrl=%2FAccount%2FLogin%3FReturnUrl%3D%252FAccount%252FLogin%253FRetu

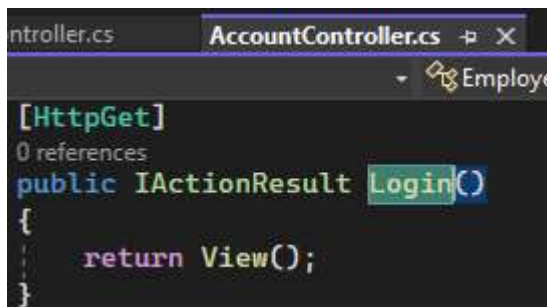
HTTP Error 404.15 - Not Found

The request filtering module is configured to deny a request where the query string is too long.

Ctrl+A->Notepad -> Paste:

[illegible]

Тези повторения са, защото приложението е започнало безкраен цикъл. Това е така, защото току-що в Startup приложихме Authorize глобално, което включва и AccountController/Login.

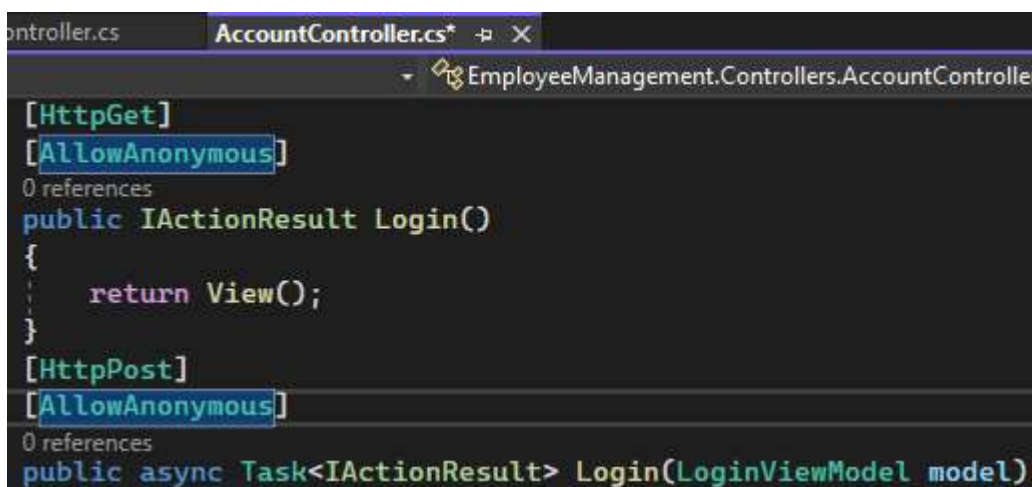


```

[HttpGet]
0 references
public IActionResult Login()
{
    return View();
}

```

Потребителят хем трябва да се впише, хем се изисква ауторизация (да бъде вече вписан за това) за това. Затова постоянно прави Redirect to URL -> Login, а когато стигне Login, се изисква пак препращане към същия адрес, докато не стане прекалено дълъг и сървърът даде грешка при обработката. За да се поправи тази грешка, просто добавяме [AllowAnonymous] -> Login:

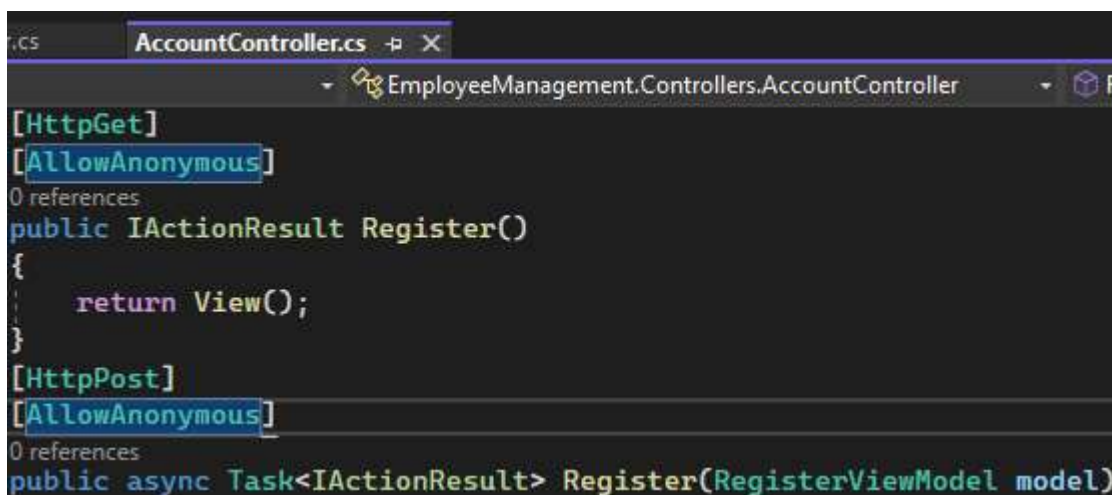


```

[HttpGet]
[AllowAnonymous]
0 references
public IActionResult Login()
{
    return View();
}
[HttpPost]
[AllowAnonymous]
0 references
public async Task<IActionResult> Login(LoginViewModel model)

```

Както и пред Register, за да може новите потребители да имат този достъп.



```

[HttpGet]
[AllowAnonymous]
0 references
public IActionResult Register()
{
    return View();
}
[HttpPost]
[AllowAnonymous]
0 references
public async Task<IActionResult> Register(RegisterViewModel model)

```

Това е кодът, с който създадохме филтър и политика за ауторизация:

Apply Authorize Attribute Globally

```
public void ConfigureServices(IServiceCollection services)
{
    // Other Code

    services.AddMvc(config => {
        var policy = new AuthorizationPolicyBuilder()
            .RequireAuthenticatedUser()
            .Build();
        config.Filters.Add(new AuthorizeFilter(policy));
    });

    // Other Code
}
```

На този етап създаваме аутентификация без допълнителни специфики, но има възможности за настройки, които ще разгледаме по-нататък.

Authorization in ASP.NET Core

